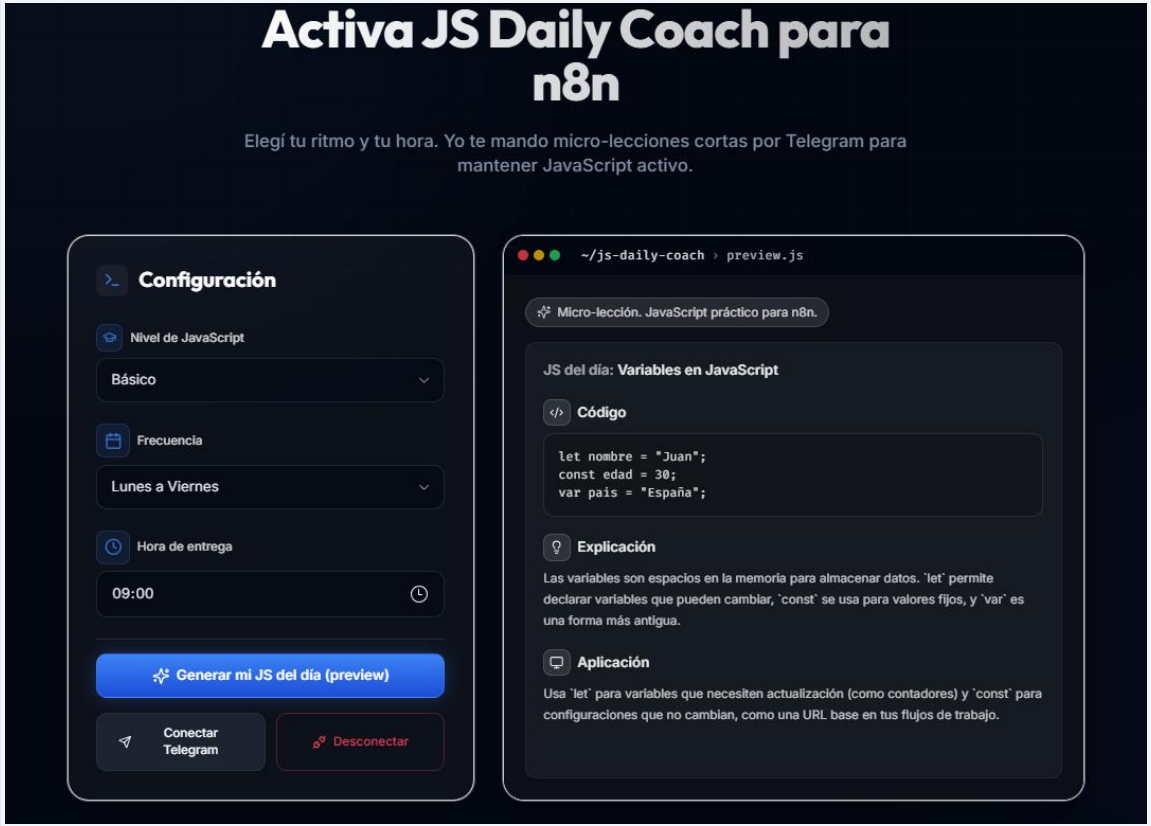
JS Daily Coach para n8n

Proceso de creación, herramientas low-code, aprendizajes y evolución del agente (Fase 4)

Jaime Blanco
IEBS

Febrero 2026



El problema

// El reto del builder de n8n: dominar lo esencial de JS

REALIDAD

El builder de n8n no programa JS todo el día, no es programador, pero necesita interpretar scripts, manipular JSON y asegurar la comunicación entre nodos y APIs

- **La sintaxis se enfría sin repetición.**
- **Impacto directo en nodos Code y HTTP Request.**
- **Riesgo: copiar código a ciegas sin entender.**
- **Repetición breve para interiorizar sintaxis, estructuras y uso de JS.**

Tipo de usuario

// A quién va dirigido el agente

El Aprendiz

Estudia JavaScript (básico a intermedio) y sabe que la repetición constante es la única forma de retener conceptos a largo plazo.

Input: Nivel

El Automatizador

Usa n8n regularmente pero se traba en los nodos Code. Necesita entender qué hace el código, no solo copiarlo.

Contexto: n8n

El Práctico

No programa todo el día. Busca práctica breve (micro-lecciones) y directa al grano sin cursos de 40 horas.

Tiempo: ~ 5min

Casos de uso

// ¿Para qué sirve realmente?

Transformación

Limpiar, formatear y reestructurar JSON complejos que llegan de webhooks o bases de datos desordenadas.

Limpiar y reestructurar datos

Lógica Avanzada

Condicionales que van más allá de un simple "If". Manejo de fechas y cálculos matemáticos en el flujo.

Condiciones y validaciones

Peticiones HTTP

Preparar payloads dinámicos y headers de autenticación para conectar con APIs externas.

Armar el HTTP Request

Cómo funciona

// El flujo del agente paso a paso

1

Configuración

El usuario define su nivel (Básico/Intermedio), frecuencia y hora de entrega preferida.

Web App (Configuración)

2

Generación

El servidor envía el nivel al LLM y genera la micro-lección (concepto + Código + explicación +aplicación).

Servidor + LLM (OpenAI)

3

Entrega

El usuario recibe la micro-lección formateada en el panel preview.

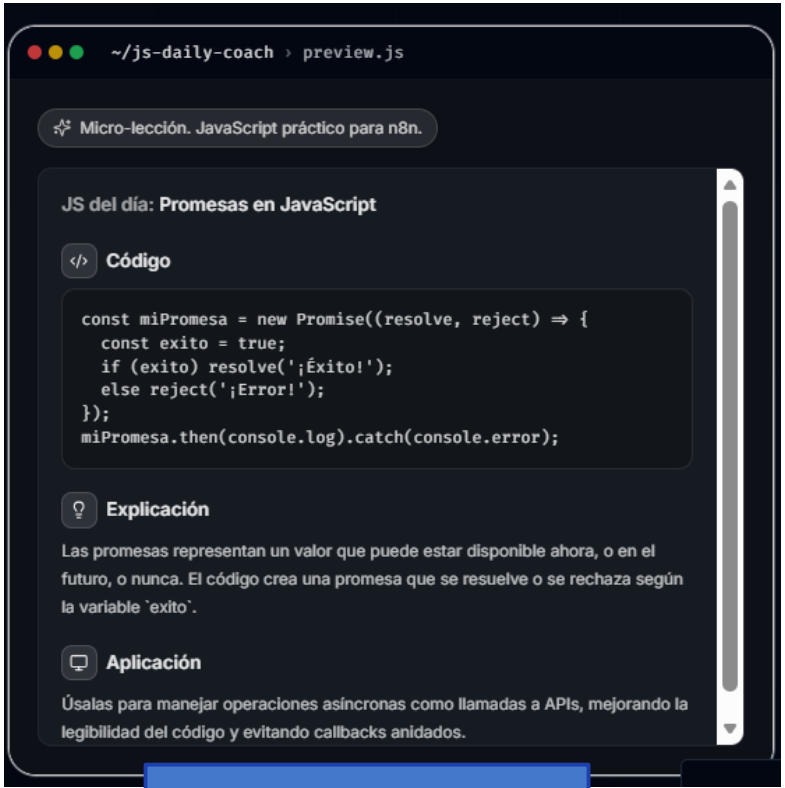
Preview en la web (MVP)

-> Entrega por Telegram: evolución propuesta en Fase 4

MVP funcional

// Evidencia del Sprint 3

- ✓ **Dificultad adaptativa**
- ✓ **Formato de lectura rápida (JS del día | Código | Explicación | Aplicación)**



Stack tecnológico

// Las herramientas detrás

ORQUESTADOR

Servidor (Node.js + Express)

Recibe los inputs, construye el prompt y hace la llamada a la API. Devuelve la micro-lección al frontend.

INTELIGENCIA

OpenAI API (gpt-4o-mini)

Genera micro-lecciones cortas y al grano con ejemplo de código, siguiendo instrucciones de formato fijo.

INTERFAZ (MVP)

Web App (React + TypeScript)

Permite configurar el agente y ver el preview del resultado en pantalla (MVP Sprint 3).

FORMATO DE SALIDA

Formato fijo (4 secciones)

JS del día / Código / Explicación / Aplicación. Lectura rápida y consistente.

Limitaciones

// ¿Dónde falla el agente?

Sin memoria

El MVP no guarda historial ni progreso. No recuerda temas vistos ni refuerza repaso espaciado.

MITIGACIÓN:
Historial + progreso + repaso espaciado

Variabilidad del LLM

Aunque existe un formato fijo, el modelo puede variar estilo/longitud o romper la estructura en algunos casos.

MITIGACIÓN:
Prompt estricto + validación de formato

Operación y alcance

Sin Telegram funcional por ahora y manejo de errores básico. Aún no cubre SQL básico ni fundamentos de APIs.

MITIGACIÓN:
Telegram + errores claros + modos SQL/APIs

Próximos pasos

// Evolución del agente

1

Entrega por Telegram

Implementar conexión real con Telegram y envíos programados. Incluir pausar/reanudar y desconectar.

Marzo 2026

2

Memoria y progreso

Guardar historial de lecciones, temas vistos y nivel. Aplicar repaso espaciado y recomendar "siguiente tema".

Abril 2026

3

Modos SQL/APIs + UX

Añadir modos "SQL básico" y "APIs + HTTP". Mejorar UX: copiar código, favoritos y resumen semanal.

Mayo 2026

Lecciones aprendidas parte 1

// Conclusiones finales

Prompting es iterativo
El primer prompt nunca es el definitivo. Ajustar la "temperatura" y las restricciones de formato fue clave.

Estructura > Contenido
Para micro-aprendizaje, el formato visual (emojis, negritas, bloques de código) importa tanto como la explicación.

Menos es más
La fricción de "entrar a un curso" mata el hábito. La entrega push en Telegram reduciría esa barrera (evolución).

Iteración rápida (Replit + ChatGPT)
Revisar archivos (React/CSS/backend) en Replit y depurar con ayuda de ChatGPT permitió probar y mejorar el MVP en ciclos cortos.

Lecciones aprendidas parte 2

// // Mensajes clave: cómo “habla” el agente

En la interfaz (UI)

Elegí tu ritmo y tu hora. Yo te mando micro-lecciones cortas por Telegram para mantener JavaScript activo.

Generar mi JS del día (preview)

Aquí aparece tu JS del día (preview).

Mensajes del sistema

Preparando tu JS del día...

¡Listo! JS del día generado
Léelo en 1 minuto y quedate con la idea clave.

Ups... no se pudo generar

Objetivo: que el agente se perciba como **coach** (cercano, directo y al grano), no solo como sistema técnico.